



Explore | Expand | Enrich

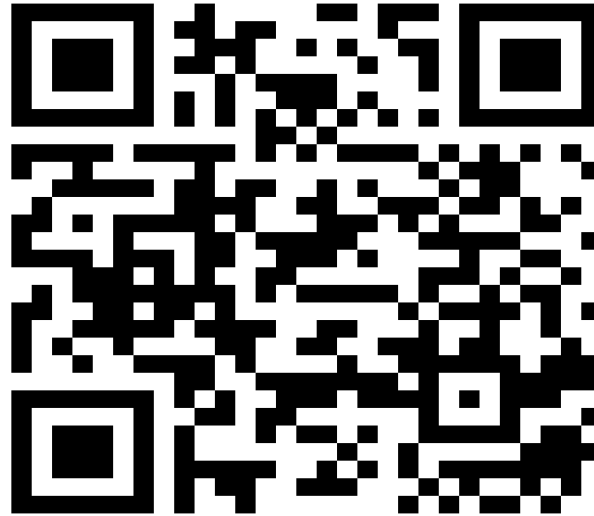
<Codemithra />TM

STOCK SPAN PROBLEM



TEST TIME ON TOWER OF HANOI

URL : <https://forms.gle/4NHVaw6w4KwLbY2P8>



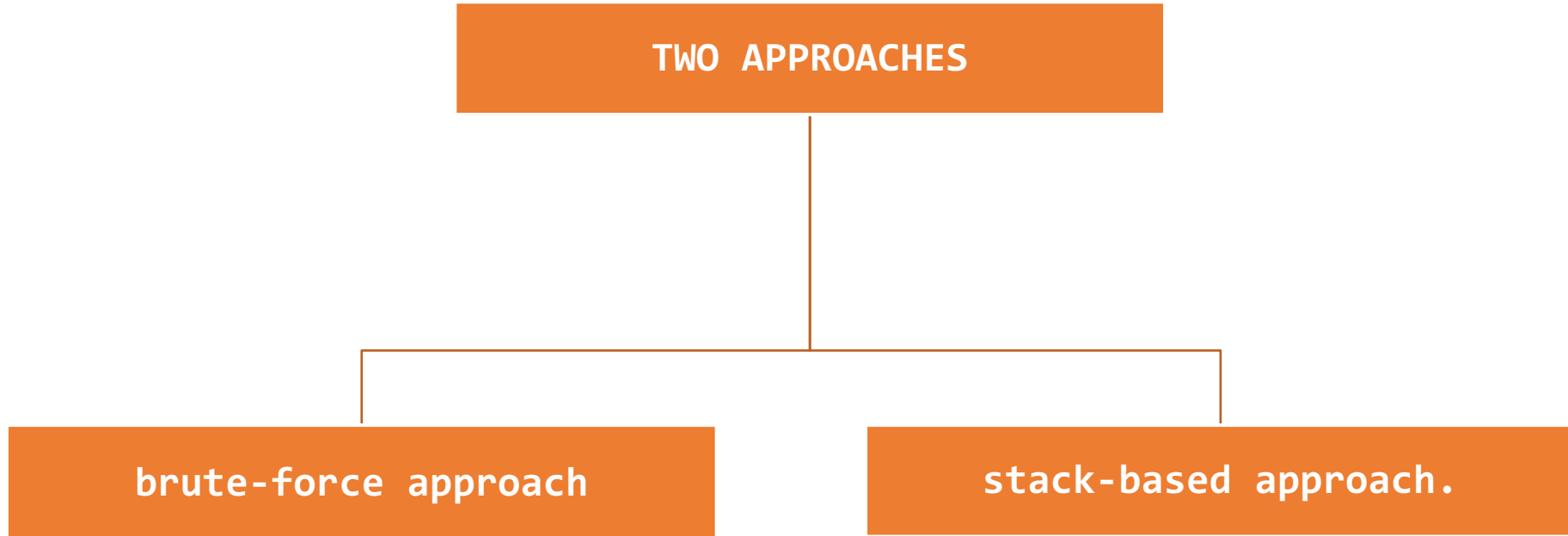
STOCK SPAN PROBLEM

EXPLANATION

- The Stock Span problem is a financial problem where we have a series of daily stock price quotes, and we need to calculate the span of stock's price for each day.
- The span of a stock's price on a given day is defined as the maximum number of consecutive days (including the current day) for which the stock price was less than or equal to the price on the current day.
- In simple terms, it measures how many days a stock had a lower or equal price leading up to the current day.

STOCK SPAN PROBLEM

APPROACHES



STOCK SPAN PROBLEM

APPROACHES 1: BRUTE FORCE APPROACH

The brute force approach to solving the Stock Span problem involves a straightforward, iterative method.

STEP-BY-STEP EXPLANATION:

Start with the first day's stock price and initialize its span as 1 because there are no previous days to compare with.

For each subsequent day (starting from the second day):

- ❖ Initialize the span for the current day as 1.
- ❖ Compare the stock price of the current day with the stock prices of all the previous days (back to day 1).



STOCK SPAN PROBLEM

APPROACHES 1: BRUTE FORCE APPROACH

- ❖ Increment the span for the current day for each previous day where the stock price is less than or equal to the stock price of the current day.
- ❖ Continue this process for each day, building the span values for the entire series of stock prices.

The final result is an array of span values, where each span value represents how many consecutive days the stock price was less than or equal to the price on the current day.

STOCK SPAN PROBLEM

APPROACHES 2: STACK BASED APPROACH

The stack based approach to solving the Stock Span problem is a more efficient and elegant solution that leverages a stack data structure.

STEP BY STEP EXPLANATION

Start with the first day's stock price and initialize its span as 1 because there are no previous days to compare with.

Create an empty stack to store the indices of previously seen stock prices.

STOCK SPAN PROBLEM

APPROACHES 2: STACK BASED APPROACH

For each subsequent day (starting from the second day):

- ❖ Initialize the span for the current day as 1.
- ❖ While the stack is not empty and the stock price of the current day is greater than or equal to the stock price at the index stored at the top of the stack:
- ❖ Pop elements from the stack until a smaller stock price is encountered.
- ❖ Increment the span for the current day during each pop operation.

STOCK SPAN PROBLEM

APPROACHES 2: STACK BASED APPROACH

Calculate the span for the current day using the difference between the current index and the index at the top of the stack. If the stack is empty, the span is simply the current index plus 1.

- ❖ Push the current index onto the stack.

Continue this process for each day, efficiently calculating the span values for the entire series of stock prices.

The final result is an array of span values, where each span value represents how many consecutive days the stock price was less than or equal to the price on the current day.

STOCK SPAN PROBLEM

SOLVING THE STOCK SPAN PROBLEM-BRUTE-FORCE APPROACH

The brute-force approach involves iterating through the stock prices day by day and, for each day, counting the number of consecutive days with lower or equal prices

STOCK SPAN PROBLEM

```
import java.util.Arrays;

public class Main {
    public int[] calculateSpan(int[]
stockPrices) {
        int n = stockPrices.length;
        int[] span = new int[n];
        span[0] = 1; // The span of the first
day is always 1.

        for (int i = 1; i < n; i++) {
            span[i] = 1; // Initialize the span
for the current day.
            int j = i - 1;
            while (j >= 0 && stockPrices[i] >=
stockPrices[j]) {
                span[i]++;
                j--;
            }
        }
    }
}
```

```
    }

    return span;
}

public static void main(String[] args) {
    Main calculator = new Main();
    int[] stockPrices = {100, 80, 60, 70,
60, 75, 85};
    int[] spans =
calculator.calculateSpan(stockPrices);

    System.out.println("Stock Prices: " +
Arrays.toString(stockPrices));
    System.out.println("Stock Spans: "
+ Arrays.toString(spans));
}
}
```



STOCK SPAN PROBLEM

TIME AND SPACE COMPLEXITY

The time complexity of the code is $O(n^2)$ in the worst case, where n is the number of stock prices, due to the nested loop used to calculate spans.

The space complexity is $O(n)$ as it requires additional memory to store the span values for each day.

STOCK SPAN PROBLEM

SOLVING THE STOCK SPAN PROBLEM-STACK BASED APPROACH

The stack-based approach is more efficient and leverages a stack data structure to keep track of the indices of previously seen stock prices.

STOCK SPAN PROBLEM



```
import java.util.Stack;
import java.util.Arrays;
public class Main {
    public int[] calculateSpan(int[]
stockPrices) {
        int n = stockPrices.length;
        int[] span = new int[n];
        Stack<Integer> stack = new Stack<>();
        span[0] = 1;
        stack.push(0);
        for (int i = 1; i < n; i++) {
            while (!stack.isEmpty() && stockPrices[i] >=
stockPrices[stack.peek()]) {
                stack.pop();
            }
            span[i] = stack.isEmpty() ? i + 1 : i -
stack.peek();
        }
    }
}
```

```
// Push the current index onto the stack.
        stack.push(i);
    }

    return span;
}

public static void main(String[] args) {
    Main calculator = new Main();
    int[] stockPrices = {100, 80, 60, 70,
60, 75, 85};
    int[] spans =
calculator.calculateSpan(stockPrices);

    System.out.println("Stock Prices: " +
Arrays.toString(stockPrices));
    System.out.println("Stock Spans:   " +
Arrays.toString(spans));
}
}
```



STOCK SPAN PROBLEM

TIME AND SPACE COMPLEXITY

Time Complexity: $O(N)$, where N is the number of stock prices. It iterates through the prices once.

Space Complexity: $O(N)$ for the stack data structure.

STOCK SPAN PROBLEM

BEST METHOD FOR SOLVING THE STOCK SPAN PROBLEM.

1. The stack-based approach is considered the best method for solving the Stock Span problem.
2. It offers improved efficiency in terms of time complexity.
3. This approach avoids redundant calculations by efficiently tracking previous stock price indices using a stack.
4. In contrast, the brute-force approach has a higher time complexity due to its reliance on nested loops.
5. The brute-force approach also performs unnecessary comparisons, making it less efficient for this problem.



INTERVIEW QUESTIONS

1. What is the Stock Span problem, and why is it important in finance?

Answer: The Stock Span problem involves calculating the span of a stock's price for each day, where the span of a day is the number of consecutive days immediately before the current day on which the stock price is less than or equal to the current day's price. It is important in finance for analyzing the performance and volatility of stocks.

INTERVIEW QUESTIONS

2.What are the key differences between the brute-force approach and the stack-based approach for solving the Stock Span problem?

Answer: The stack-based approach is preferred as it offers better time complexity by avoiding redundant calculations and efficiently tracking previous stock price indices using a stack. In contrast, the brute-force approach involves nested loops and performs unnecessary comparisons, resulting in higher time complexity.

INTERVIEW QUESTIONS

3.Explain the stack-based approach to solving the Stock Span problem with a code example.

Answer: The stack-based approach uses a stack to keep track of indices of stock prices. It iterates through the stock prices, efficiently popping elements from the stack when necessary and calculating spans based on the difference between the current index and the index at the top of the stack. This approach optimally solves the problem.

INTERVIEW QUESTIONS

4. What is the time complexity of the stack-based solution for the Stock Span problem, and why is it efficient?

Answer: The stack-based solution has a time complexity of $O(n)$, where n is the number of stock prices. It is efficient because it avoids redundant calculations and optimally tracks previous stock price indices using a stack, resulting in a linear time complexity.

Answer: The time complexity is $O(2^n)$, where n is the number of disks.

INTERVIEW QUESTIONS

5. In what scenarios would you use the Stock Span problem to analyze stock market data?

Answer: The Stock Span problem is useful for analyzing stock market data when you want to understand the consecutive days on which a stock's price showed consistent growth or decline. It helps identify trends, assess market sentiment, and make informed investment decisions.

INTERVIEW QUESTIONS

6.Can you provide a real-world application of the Stock Span problem outside of finance?

Answer: The Stock Span problem can be applied in various scenarios, such as analyzing the consecutive days of temperature increase or decrease in weather data. It helps identify trends and patterns in temperature fluctuations, which can be valuable for weather forecasting and climate analysis.



<https://learn.codemithra.com>



THANK YOU